SRM MADURAI

COLLEGE FOR ENGINEERING AND TECHNOLOGY

Pottapalayam, Sivagangai District - 630 612.

DEPARTMENT OF

Approved by All India Council for Technical Education

and Affiliated to Anna University, Chennai.

SRM MADURAI

COLLEGE FOR ENGINEERING AND TECHNOLOGY

Approved by AICTE, New Delhi | Affiliated to Anna University, Chennai

DEPARTMENT OF _____

SRM MADURAI COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM MCET

RECORD

Name : .....

Roll no : .....

Course : .....

SRM MADURAI

COLLEGE FOR ENGINEERING AND TECHNOLOGY

Approved by AICTE, New Delhi | Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

Register no.

Certified to be the bonafide record of work done by .....

of ..... Semester B.E / B.Tech ..... course in the .....

Laboratory in SRM Madurai College for Engineering and Technology, Pottapalayam,

during the academic year .....

Staff in Charge

Head of the Department

Submitted for the University Practical Examination held on ..... at,

SRM Madurai College for Engineering and Technology, Pottapalayam.

Internal Examiner

External Examiner

GENERAL INSTRUCTIONS

1. The Record should be written neatly in ink pen or ball point pen.

2. **The Record should contain**

   (I) Experiment No. and Date

   (II) Title of the Experiment

   (III) Aim of the Experiment

   (IV) Procedure

   (V) Calculation

   (VI) Result

3. Neat Sketches of apparatus used, tabulations and calculations are to be given on the left-hand page of the Record.

4. Labelled diagrams in pencil should be drawn neatly whenever necessary.

CONTENTS

S.No	Date	Name of the Experiment	Pg.no	Date of Submission	Marks	Staff Signature

CONTENTS

S.No	Date	Name of the Experiment	Pg.no	Date of Submission	Marks	Staff Signature

STAFF INCHARGE

EX NO: 1A WORKING WITH NUMPY ARRAYS

NUMPY:

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices. NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely. NumPy stands for Numerical Python.

It is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

AIM

Write a Python program to demonstrate basic array characteristics.

ALGORITHM

Step1: Start

Step2: Import numpy module

Step3: Print the basic characteristics of array Step4:

Stop

PROGRAM

```
importnumpy as np
 # Creating array object arr
= np.array( [[ 1, 2, 3],
[ 4, 2, 5]] )
```

```
# Printing type of arr object
print("Array is of type: ", type(arr)) #
Printing array dimensions (axes)
print("No. of dimensions: ", arr.ndim) #
Printing shape of array print("Shape of
array: ", arr.shape)
  # Printing size (total number of elements) of array
print("Size of array: ", arr.size)

# Printing type of elements in array
print("Array stores elements of type: ", arr.dtype)
```

OUTPUT

RESULT

Thus, the python program working with NumPy array has been implemented and executed successfully.

EX.NO :1B PROGRAM TO PERFORM ARRAY SLICING

SLICING:

Similar to Python lists, numpy arrays can be sliced. Since arrays may be multidimensional, we must specify a slice for each dimension of the array

AIM

Write a Python Program to Perform Array Slicing.

ALGORITHM

Step1: Start

Step2: import numpy module

Step3: Create an array and apply the slicing operator Step4:

Print the output

Step5: Stop

PROGRAM

```
Import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])
print(a)
print("After slicing")
print(a[1:])
```

OUTPUT

RESULT

Thus, the python program to perform array slicing has been implemented and executed successfully.

EX NO: 1C PROGRAM TO PERFORM ARRAY SLICING

AIM

Write a Python Program to Perform Array Slicing.

ALGORITHM

Step1: Start

Step2: import numpy module

Step3: Create an array and apply the slicing operator Step4:

Print the output

Step5: Stop

PROGRAM

```
# array to begin with import
numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])

print('Our array is:' ) print(a)
 # this returns array of items in the second column
print('The items in the second column are:' )
print(a[...,1])
print('\n' )

# Now we will slice all items from the second row
print ('The items in the second row are:' )
print(a[1,...])
print('\n' )

# Now we will slice all items from column 1 onwards
print('The items column 1 onwards are:' )
print(a[...,1:])
```

OUTPUT:

RESULT

Thus, the python program to perform array slicing has been implemented and executed successfully.

EX NO :2A WORKING WITH PANDAS DATA FRAME

PANDAS:

It is a Python library. Pandas is used to analyze data. A Pandas DataFrame is a 2 dimensional datastructure, like a 2 dimensional array, or a table with rows and columns.Pandas DataFrame can becreatedfrom thelists, dictionary andfrom a list ofdictionaryetc.

CREATE A DATAFRAME USING A LIST OF ELEMENTS

AIM

Write a program to create a dataframe using a list of elements.

ALGORITHM

Step1: Start

Step2: import numpy and pandas module Step3:

Create a dataframe using list of elements Step4:

Print the output

Step5: Stop

PROGRAM

```
import pandas as pd

import pandas as pd #

list of strings

lst = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
# Calling DataFrame constructor on list df

= pd.DataFrame(lst)

print(df)
```

OUTPUT

RESULT

Thus, the python program for dataframe using list of elements has been implemented and executed successfully.

EX NO: 2B CREATE A DATAFRAME USING THE DICTIONARY

DATAFRAME:

To create DataFrame from dict of narray/list, all the narray must be of same length. If index is passed then the length index should be equal to the length of arrays. If no index is passed, then by default, index will be range(n) where n is the array length.

AIM

Write a program to create a data frame using dictionary of elements.

ALGORITHM

Step1: Start

Step2: import numpy and pandas module Step3:

Create a data frame using the dictionary Step4:

Print the output

Step5: Stop

PROGRAM

```
import pandas as pd
 # intialise data of lists.
data = {'Name':['Tom', 'nick', 'krish', 'jack'], 'Age':[20,
        21, 19, 18]}
 # Create DataFrame
df = pd.DataFrame(data) #
Print the output. print(df)
```

OUTPUT:

RESULT

Thus, the python to create data frame using dictionary program has been implemented and executed successfully

EX NO: 2C COLUMN SELECTION

Column Selection

A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. We can perform basic operations on rows/columns like selecting, deleting, adding, and renaming.

Column Selection: In Order to select a column in Pandas DataFrame, we can either access the columns by calling them by their columns name.

AIM

Write a program to select a column from dataframe.

ALGORITHM

Step1: Start

Step2: import pandas module

Step3: Create a dataframe using the dictionary

Step4: Select the specific columns and print the output Step5:

Stop

PROGRAM

```
import pandas as pd
 # Define a dictionary containing employee data
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age':[27, 24, 22, 32],        'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'], 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
 # Convert the dictionary into DataFrame df
= pd.DataFrame(data)
print(df)
 # select two columns
print(df[['Name', 'Qualification']]
```

OUTPUT:

RESULT

Thus, the python program for column selection has been implemented and executed successfully.

EX NO: 2D CHECKING FOR MISSING VALUES USING ISNULL() AND NOTNULL()

In order to check missing values in Pandas DataFrame, we use a function isnull() and notnull(). Both function help in checking whether a value is NaN or not.These function can also be used in Pandas Series in order to find null values in a series.

AIM

Write a program to check the missing values from the data frame.

ALGORITHM

Step1: Start

Step2: import pandas module

Step3: Create a dataframe using the dictionary

Step4: Check the missing values using isnull() function

Step5: print the output

Step6: Stop

PROGRAM

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
importnumpy as np
# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}

# creating a dataframe from list df =
pd.DataFrame(dict)
 # using isnull() function
df.isnull()
```

OUTPUT:

RESULT

Thus the python program checking for missing value using is null() and nonull() has been implemented and executed successfully.

EX NO: 2E DROPPING MISSING VALUES USING DROPNA()

In order to drop a null values from a data frame, we used dropna() function this function drop Rows/Columns of datasets with Null values in different ways.

AIM

Write a program to drop rows with at least one Nan value (Null value)

ALGORITHM

Step1: Start

Step2: import pandas module

Step3: Create a dataframe using the dictionary Step4:

Drop the null values using dropna() funtion Step5:

print the output

Step6: Stop

PROGRAM

Drop rows with at least one Nan value (Null value)

```
# importing pandas as pd
import pandas as pd
# importing numpy as np
import numpy as np
# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],'Second Score': [30, np.nan, 45, 56],
        'Third Score':[52, 40, 80, 98],
        'Fourth Score':[np.nan, np.nan, np.nan, 65]} #
        creating a dataframe from dictionary
        df = pd.DataFrame(dict) #
using dropna()function

df.dropna()
```

OUTPUT:

RESULT

Thus, the python program for Drop missing values has been implemented and executed successfully.

EX NO: 3A BASIC PLOTS USING MATPLOTLIB

MATPLOTLIB:

It is a Python library that helps in visualizing and analyzing the data and helps in better understanding of the data with the help of graphical, pictorial visualizations that can be simulated using the matplotlib library. Matplotlib is a comprehensive library for static, animated and interactive visualizations.

AIM

Write a python program to create a simple plot using plot () function.

ALGORITHM

Step1: Define the x-axis and corresponding y-axis values as lists. Step2:

Plot them on canvas using. plot () function.

Step3: Give a name to x-axis and y-axis using. xlabel() and .ylabel() functions.

Step4: Give a title to your plot using. title () function.

Step5: Finally, to view your plot, we use. show () function.

Step6: Stop

PROGRAM

```
Import matplotlib.pyplot as plt #
 x axis values
x = [1,2,3]

# corresponding y axis values y
= [2,4,1]
 # plotting the points
plt.plot(x, y)
# naming the x axis
plt.xlabel('x - axis') #
naming the y axis
plt.ylabel('y - axis')
# giving a title to my graph
plt.title('My first graph!')
# function to show the plot
plt.show()
```

OUTPUT:

RESULT

Thus the python program for basic Matplotlib has been implemented and executed successfully.

EX NO: 3B COMPUTE THE X AND Y COORDINATES AND CREATE A PLOT

AIM

Write a python program to create a plot by computing the x and y coordinates.

ALGORITHM

Step1: Compute the x and y coordinates for points on a sine curve Step2:

Plot the points using matplotlib

Step3:Display the output

Step4: Stop

PROGRAM

```
importnumpyasnp
importmatplotlib.pyplotasplt x
=np.arange(0, 3*np.pi, 0.1) y
=np.sin(x)

plt.plot(x, y)
plt.show()
```

OUTPUT

RESULT

Thus the python program to compute X and Y coordinates has been implemented and executedsuccessfully.

EX NO: 3C DRAWING MULTIPLE LINES USING PLOT FUNCTION

AIM

Write a python program to draw multiple lines using plot() function.

ALGORITHM

Step1: Compute the x and y coordinates for points on a sine and cosine curve

Step2: Plot the points using matplotlib

Step3:Display the output

Step4: Stop **PROGRAM**

```
importnumpy as np
importmatplotlib.pyplot as plt
# Compute the x and y coordinates for points on sine and cosine curves x
= np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x) y_cos
= np.cos(x)
# Plot the points using matplotlib
plt.plot(x, y_sin)
plt.plot(x, y_cos) plt.xlabel('x
axis label') plt.ylabel('y axis
label') plt.title('Sine and
Cosine') plt.legend(['Sine',
'Cosine']) plt.show()
```

OUTPUT

RESULT

Thus the python program multiple line using plot functiont has been implemented and executed successfully.

Ex No: 3D BASIC PLOT USING MATPLOTLIB

AIM

Write a python program for basic plot using matplotlib

ALGORITHM

Step1: import the library

Step2: Plot the points using matplotlib Step3:

Display the output

Step4: Stop

PROGRAM

Line plot :

```
from matplotlib import pyplot as plt x
= [5, 2, 9, 4, 7]
y = [10, 5, 8, 4, 2]

plt.plot(x,y) plt.show()
```

Bar plot :

```
from matplotlib import pyplot as plt x
= [5, 2, 9, 4, 7]
y = [10, 5, 8, 4, 2]

plt.bar(x,y) plt.show()
```

Histogram :

```
from matplotlib import pyplot as plt y
y= [10, 5, 8, 4, 2]
plt.hist(y) plt.show()
```

Scatter Plot :

```
from matplotlib import pyplot as plt x
= [5, 2, 9, 4, 7]
y = [10, 5, 8, 4, 2]
plt.scatter(x, y)
plt.show()
```

OUTPUT

RESULT

Thus the python program for basic plot using Matplotlib has been implemented and executed successfully.

EX NO:4A CONDITIONAL FREQUENCY DISTRIBUTION

AIM

To write a python program to show the conditional Frequency distribution

ALGORITHM

Step 1: Start

Step 2: Import Pandas, Numpy And Nltk

Step 3: List The Items As ‘F’ For Fruits And ’V’ For Vegetables

Step 4: Display The Frequency Of Each Items In The List

Step 5: Stop

PROGRAM:

```
importnumpyasnp# linear algebra
importpandasaspd# data processing, CSV file I/O (e.g. pd.read_csv)

importnltk
items = ['apple', 'apple', 'kiwi', 'cabbage', 'cabbage', 'potato']
nltk.FreqDist(items)
c_items= [('F','apple'), ('F','apple'), ('F','kiwi'), ('V','cabbage'), ('V','cabbage'), ('V','potato') ]
cfd=nltk.ConditionalFreqDist(c_items)
cfd.conditions()
cfd.plot() cfd['V']
```

OUTPUT

RESULT

Thus the python program for conditional frequency distribution has been implemented and executed successfully.

EX NO: 4B FREQUENCY OF WORDS, OF A PARTICULAR GENRE, IN BROWN CORPUS.

AIM : To write a program to find the frequesncy of words, particular genre and in brown corpus

ALGORITHM

Step 1: Start

Step 2: Import All Necessary Libraries

Step 3: Display The Frequency Of Each Items In The List

Step 4:Setting Cumulative Argument Value To True.

Step 5: Stop

PROGRAM

```
fromnltk.corpusimport brown
cfd= nltk.ConditionalFreqDist([ (genre, word) for genre inbrown.categories() for word inbrown.words(categories=genre) ]) cfd
cfd.conditions()
cfd.tabulate(conditions=['government', 'humor', 'reviews'],samples=['leadership', 'worship', 'hardship'])
cfd.plot(conditions=['government', 'humor', 'reviews'],samples=['leadership', 'worship', 'hardship'])
cfd.tabulate(conditions=['government', 'humor', 'reviews'], samples=['leadership', 'worship', 'hardship'], cumulative =True)
news_fd=cfd['news']
news_fd.most_common(3)
news_fd['the']
```

OUTPUT

RESULT

Thus the python program frequency of words, of a particular genre, in brown corpus has been implemented and executed successfully.

EX NO: 4C FREQUENCY OF LAST CHARACTER APPEARING IN ALL NAMES ASSOCIATED WITH MALES AND FEMALES RESPECTIVELY AND COMPARES THEM

AIM

To write a python program frequency of last character appearing in all names associated with malesand females respectively and compares them.

ALORITHM

Step 1: Start

Step 2: Import All Necessary Libraries

Step 3: Display The Frequency Of Each Items In The List

Step 4: Plot

Step 5: Stop

PROGRAM

```
fromnltk.corpusimport names
nt= [(fid.split('.')[0], name[-1])    for fid innames.fileids() for name innames.words(fid) ]
cfd2 =nltk.ConditionalFreqDist(nt)
cfd2['female']['a']
cfd2['male']['a'] cfd2['female'] >
cfd2['male']
cfd2.tabulate(samples=['a', 'e'])
cfd2.plot()
```

OUTPUT

RESULT

Thus the python program frequency of last character appearing in all names associated with malesand females respectively and compares them has been implemented and executed successfully.

EX NO: 4D AVERAGE OF LIST USING LOOP

AIM

To write a python program for finding a average of list using loop.

ALGORITHM

Step 1: Start

Step 2: Define A Class Cal_Average

Step 3: Sum_Num = Sum_Num + T Step

4: Avg = Sum_Num / Len(Num) Step 5:

Stop

PROGRAM:

```
defcal_average(num):
sum_num = 0 for
t in num:
sum_num = sum_num + t avg =
sum_num / len(num) returnavg
print("The average is", cal_average([18,25,3,41,5]))
```

OUTPUT:

RESULT

Thus the python program finding a average of list using loop has been implemented and executedsuccessfully.

EX NO: 4E AVERAGE OF LIST USING BUILT IN FUNCTIONS

AIM

To write a python program to find the average of list using built in functions.

ALGORITHM

STEP 1: Start STEP

STEP 2: define a list

STEP 3: avg = sum(number_list)/len(number_list)

STEP 4:printavg

STEP 5: Stop

PROGRAM

```
def cal_average(num):
    sum_num = 0
    for t in num:
        sum_num = sum_num + t
    avg = sum_num / len(num)
    return avg


print("The average is", cal_average([18, 25,
3, 41, 5]))
```

OUTPUT:

RESULT

Thus the python program finding a average of list using built in functions has been implemented andexecuted successfully.

Ex No: 4F AVERAGE OF LIST USING MEAN FUNCTION

AIM

To write a python program to find the average of list using mean function.

ALGORITHM

Step 1: Start

Step 2: Define A List

Step 3: Import Mean From Statistics

Step 4: Avg = Mean(Number_List) Step

5: Printavg

Step 6: Stop

PROGRAM

```
from statistics import mean
number_list = [45, 34, 10, 36, 12, 6, 80] avg
= mean(number_list)
print("The average is ", round(avg,2))
```

OUTPUT:

RESULT

Thus the python program average of list using mean function has been implemented and executedsuccessfully.

EX NO: 4G AVERAGE OF LIST USING NUMPY LIBRARY

AIM

To write a python program to find the average of list using numpy library.

ALGORITHM

Step 1: Start

Step 2: Import Mean From Numpy

Step 3: Define A List

Step 4: Avg = Mean(Number_List)

Step 5:Printavg

Step 6: Stop

PROGRAM

```
From numpy import mean
number_list = [45, 34, 10, 36, 12, 6, 80] avg
= mean(number_list)
print ("The average is ", round(avg,2))
```

OUTPUT:

RESULT

Thus the python program average of list using numpy library has been implemented and executed successfully.

EX NO: 4H VARIANCE OF SAMPLE SET

AIM

To write a python program to show variance of sample set.

ALGORITHM

Step 1: Start

Step 2: Import Statistics

Step 3: Define A List

Step 4: Print Statistics.Variance(Sample))

Step 5: Stop

PROGRAM

```
import statistics
sample = [2.74, 1.23, 2.63, 2.22, 3, 1.98]
print("Variance of sample set is % s" , statistics.variance(sample))
```

OUTPUT :

RESULT

Thus the python program to show variance of sample set has been implemented and executed successfully.

EX NO: 4I VARIANCE ON A RANGE OF DATA-TYPES

AIM

To write a python program to show variance on a range of data-types.

ALGORITHM

Step 1: Start

Step 2: Import All Necessary Libraries Step

3: Define Samples

Step 4: Print Variance Of Sample Step

5: Stop

PROGRAM

```
from statistics import variance from
fractions import Fraction as fr
sample1 = (1, 2, 5, 4, 8, 9, 12)
sample2 = (-2, -4, -3, -1, -5, -6)
sample3 = (-9, -1, -0, 2, 1, 3, 4, 19)
sample4 = (fr(1, 2), fr(2, 3), fr(3, 4),fr(5, 6), fr(7, 8))
sample5 = (1.23, 1.45, 2.1, 2.2, 1.9)
print("Variance of Sample1 is ",variance(sample1))
print("Variance of Sample2 is ",variance(sample2))
print("Variance of Sample3 is ",variance(sample3))
print("Variance of Sample4 is ", variance(sample4))
print("Variance of Sample5 is ",variance(sample5))
```

OUTPUT

RESULT

Thus the python program to show variance on a range of data-types has been implemented and executed successfully.

EX NO: 4J STATISTICS

AIM

To write a python program to show statistics.

ALGORITHM

Step 1: Start

Step 2: Import Statistics

Step 3: Define A List

Step 4: M=Statistics.Mean(Sample)

Step 5: Stop

PROGRAM

```
import statistics
sample = (1, 1.3, 1.2, 1.9, 2.5, 2.2)
m = statistics.mean (sample)
print("Variance of Sample set is ",statistics.variance(sample, xbar = m))
```

OUTPUT

RESULT

Thus the python program to show statistics has been implemented and executed successfully.

EX NO: 5A CREATE NORMAL CURVE

`

AIM

To write a python program to create a normal curve.

ALGORITHM

STEP 1: Start

STEP 2: import all necessary packages

STEP 3: create distribution

STEP 4: visualize the distribution

STEP 5: Stop

PROGRAM

```
From scipy.stats import norm
importnumpy as np
importmatplotlib.pyplot as plt
importseaborn as sb
data = np.arange(1,10,0.01)
pdf = norm.pdf(data , loc = 5.3 , scale = 1 )
sb.set_style('whitegrid')
sb.lineplot(data, pdf , color = 'black')
plt.xlabel('Heights')
plt.ylabel('Probability Density')
```

OUTPUT

RESULT

Thus the python program to create a normal curve has been implemented and executed successfully.

EX NO: 5B CORRELATION AND SCATTER PLOTS

CORRELATION:

Correlation means an association. It is a measure of the extent to which two variables are related.

AIM:

To write a python program correlation and scatter plots.

ALGORITHM:

Step 1: Importing the libraries.

Step 2: Finding the Correlation between two variables.

Step 3: Plotting the graph. Here we are using scatter plots. A scatter plot is a diagram where each value in the data set is represented by a dot. Also, it shows a relationship between two variables.

PROGRAM:

```
importsklearn importnumpy as np
importmatplotlib.pyplot as plt
import pandas as pd
y = pd.Series([1, 2, 3, 4, 3, 5, 4])

x = pd.Series([1, 2, 3, 4, 5, 6, 7])

correlation = y.corr(x)
print(correlation)
plt.scatter(x, y)
 # This will fit the best line into the graph

plt.plot(np.unique(x), np.poly1d(np.polyfit(x, y, 1)) (np.unique(x)), color='red')
```

OUTPUT:

RESULT:

Thus the python program to correlation and scatter plots has been implemented and executed successfully.

5B SCATTER PLOT:

Scatter plot is a graph of two sets of data along the two axes. It is used to visualize the relationship between the two variables.

In python matplotlib, the scatterplot can be created using the pyplot.plot() or the pyplot.scatter(). Using these functions, you can add more feature to your scatter plot, like changing the size, color orshape of the points.

i)SIMPLE SCATTER PLOT

AIM:

To write a python program simple scatter plots.

ALGORITHM:

Step 1: Importing the libraries.

Step 2: Finding the Correlation between two variables.

Step 3: Plotting the graph. Here we are using scatter plots. A scatter plot is a diagram where each value in the data set is represented by a dot. Also, it shows a relationship between two variables.

PROGRAM:

```
x = range(50)
y = range(50) + np.random.randint(0,30,50)
plt.scatter(x, y)
plt.rcParams.update({'figure.figsize':(10,8), 'figure.dpi':100})
plt.title('Simple Scatter plot')
plt.xlabel('X - value')
plt.ylabel('Y - value')
plt.show()
```

OUTPUT:

RESULT

Thus the python program for simple scatter Plot has been implemented and executed successfully.

ii)SIMPLE SCATTER PLOT WITHCOLORED POINTS

AIM:

To write a python program Simple Scatterplot with colored points.

ALGORITHM:

Step 1: Importing the libraries.

Step 2: Finding the Correlation between two variables.

Step 3: Plotting the graph. Here we are using scatter plots. A scatter plot is a diagram where each value in the data set is represented by a dot. Also, it shows a relationship between two variables.

PROGRAM:

```
x = range(50)

y = range(50) + np.random.randint(0,30,50)
plt.rcParams.update({'figure.figsize':(10,8), 'figure.dpi':100}) plt.scatter(x,
y, c=y, cmap='Spectral')

plt.colorbar() plt.title('Simple
Scatter plot') plt.xlabel('X -
value')

plt.ylabel('Y - value')
plt.show()
```

OUTPUT:

RESULT:

Thus the python program Simple Scatterplot with colored points has been implemented andexecuted successfully.

EX NO: 5C **CORRELATION COEFFICIENT**

NUMPY CORRELATION CALCULATION

AIM:

To write a program to calculate the correlation coefficient.

ALGORITHM:

STEP 1: Import the numpy packages.

STEP 2: Define two NumPy arrays. Call them x and y

STEP3: Call np.corrcoef() with both arrays as arguments

STEP 4: corrcoef() returns the correlation matrix, which is a two-dimensional array with the correlation coefficients.

PROGRAM:

```
importnumpy as np
x = np.arange(10, 20)
y = np.array([2, 1, 4, 5, 8, 12, 18, 25, 96, 48])
r = np.corrcoef(x, y) print(r)
```

OUTPUT:

RESULT:

Thus the python program calculate the correlation coefficient has been implemented and executed successfully.

PEARSON'S CORRELATION

AIM:

To write a program to calculate the Pearson correlation coefficient between two variables.

ALGORITHM:

Step 1: Import The Needed Packages.

Step 2: Provide The Data.

Step 3: Thepearsonr() Scipy Function Can Be Used To Calculate The Pearson's Correlation Coefficient Between Two Data Samples With The Same Length. Step

4: Display The Correlation Coefficient.

PROGRAM:

```
fromnumpy.random import randn
fromnumpy.random import seed
fromscipy.stats import pearsonr
seed(1)
data1 = 20 * randn(1000) + 100
data2 = data1 + (10 * randn(1000) + 50)
corr,_ = pearsonr(data1, data2)
print('Pearsons correlation:', corr)
```

OUTPUT:

RESULT:

Thus the python program to calculate the Pearson correlation coefficient between two variables hasbeen implemented and executed successfully.

6.REGRESSION

EX NO: 6A SIMPLE LINEAR REGRESSIONWITH SCIKIT LEARN

AIM:

To write a program simple linear regression with scikit-learn.

ALGORITHM:

Step 1: Import The Packages And Classes.

Step 2: Provide Data To Work With And Eventually Do Appropriate Transformations. Step 3: Create A Regression Model And Fit It With Existing Data.

Step 4: Check The Results Of Model Fitting To Know Whether The Model Is Satisfactory. Step 5: Apply The Model For Predictions.

PROGRAM:

```
importnumpy as np
fromsklearn.linear_model import LinearRegression x =
np.array([5, 15, 25, 35, 45, 55]).reshape((-1, 1))
y = np.array([5, 20, 14, 32, 22, 38])
model = LinearRegression().fit(x, y) r_sq
= model.score(x, y)
print('coefficient of determination:', r_sq)
y_pred = model.predict(x) print('predicted
response:', y_pred)
```

OUTPUT:

RESULT:

Thus the python program simple linear regression with scikit-learn has been implemented andexecuted successfully.

EX NO: 6B MULTIPLE LINEAR REGRESSIONWITH SCIKIT-LEARN

AIM

To write a program multiple linear regression with scikit-learn.

ALGORITHM:

Step 1:Import Packages And Classes

Step 2: Provide Data

Step 3:Create A Model And Fit It Step

4: Get Results

Step 5: Predict Response

PROGRAM:

```
importnumpy as np
fromsklearn.linear_model import LinearRegression
x = [[0, 1], [5, 1], [15, 2], [25, 5], [35, 11], [45, 15], [55, 34], [60, 35]]
y = [4, 5, 20, 14, 32, 22, 38, 43]
x, y = np.array(x), np.array(y) model =
LinearRegression().fit(x, y)

r_sq = model.score(x, y) print('coefficient of
determination:', r_sq) print('intercept:',
model.intercept_) print('slope:',
model.coef_)
y_pred = model.predict(x)
print('predicted response:', y_pred)
```

OUTPUT:

RESULT:

Thus the python program multiple linear regression with scikit-learn has been implemented and executed successfully.

EX NO: 7 Z-TEST CASE STUDIES

AIM

To Perform Z-test

ALGORITHM

Step1: Start

Step2: Import

math,numpy,statsmodels&ztest Step3: create

a list & Print the ztest list Step4: Stop

PROGRAM

```
# imports

import math

importnumpy as np

fromnumpy.random import randn

fromstatsmodels.stats.weightstats import ztest

# Generate a random array of 50 numbers having mean 110 and sd 15

# similar to the IQ scores data we assume above

mean_iq = 110

sd_iq = 15/math.sqrt(50) a

lpha =0.05

null_mean =100

data = sd_iq*randn(50)+mean_iq

# print mean and sd
```

```
print('mean=%.2f stdv=%.2f' % (np.mean(data), np.std(data)))

# now we perform the test. In this function, we passed data, in the value parameter

# we passed mean value in the null hypothesis, in alternative hypothesis we check whether the

# mean is larger

ztest_Score, p_value= ztest(data,value = null_mean, alternative='larger')

# the function outputs a p_value and z-score corresponding to that value, we compare the

# p-value with alpha, if it is greater than alpha then we do not null hypothesis

# else we reject it.

if(p_value< alpha):

    print("Reject Null Hypothesis")

else:

    print("Fail to Reject NUll Hypothesis")
```

OUTPUT

RESULT

Thus the program for Z-Test case studies has been executed and verified successfully.

EX NO: 8 T-TEST CASE STUDIES

AIM

To Perform T-test for sampling distribution.

ALGORITHM

Step1: Start

Step2: Import random &numpy

Step3: Calculate the standard deviation

Step4: Stop

PROGRAM

```
# Importing the required libraries and packages
import numpy as np
fromscipy import stats
# Defining two random distributions
# Sample Size
N = 10
# Gaussian distributed data with mean = 2 and var = 1
x = np.random.randn(N) + 2
# Gaussian distributed data with mean = 0 and var = 1
y = np.random.randn(N)
# Calculating the Standard Deviation
# Calculating the variance to get the standard deviation
var_x = x.var(ddof = 1)
var_y = y.var(ddof = 1)
# Standard Deviation
SD = np.sqrt((var_x + var_y) / 2)
```

```
print("Standard Deviation =", SD)
# Calculating the T-Statistics
tval = (x.mean() - y.mean()) / (SD * np.sqrt(2 / N))
# Comparing with the critical T-Value
# Degrees of freedom
dof = 2 * N - 2
# p-value after comparison with the T-Statistics
pval = 1 - stats.t.cdf( tval, df = dof)
print("t = " + str(tval))
print("p = " + str(2 * pval))
## Cross Checking using the internal function from SciPy Package
tval2, pval2 = stats.ttest_ind(x, y)
print("t = " + str(tval2))
print("p = " + str(pval2))
```

OUTPUT:

RESULT

Thus the program for T-test case studies has been executed and verified successfully.

EX NO: 9 ANOVA CASE STUDIES

AIM

To Perform ANOVA test.

ALGORITHM

Step1: Start

Step2: Import scipy

Step3: import statsmodels

Step4: calculate ANOVA F and p value

Step 5: Stop

PROGRAM

```
importscipy.stats as stats
# statsf_oneway functions takes the groups as input and returns ANOVA F and p value
fvalue, pvalue = stats.f_oneway(df['A'], df['B'], df['C'], df['D'])
print(fvalue, pvalue)
# 17.492810457516338 2.639241146210922e-05
# get ANOVA table as R like output
importstatsmodels.api as sm
fromstatsmodels.formula.api import ols
# Ordinary Least Squares (OLS) model
model = ols('value ~ C(treatments)', data=df_melt).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
anova_table
```

OUTPUT

RESULT

Thus the program for ANOVA case studies has been executed and verified successfully.

EX NO: 10 CROSS-VALIDATION WITH LINEAR REGRESSION

AIM :

To Perform Linear Regression

ALGORITHM

Step1: Start

Step2: Import numpy,pandas,seaborn,matplotlib & sklearn

Step3: calculate linear regression using the appropriate functions

Step4: display the result

Step 5: Stop

PROGRAM

```
# import all libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import scale
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline

import warnings # supress warnings
warnings.filterwarnings('ignore')
# import Housing.csv
housing = pd.read_csv('../input/cross-val/Housing.csv')
```

```
housing.head()


# number of observations
len(housing.index)
# filter only area and price
df = housing.loc[:, ['area', 'price']]
df.head()
# recaling the variables (both)
df_columns = df.columns
scaler = MinMaxScaler()
df = scaler.fit_transform(df)

# rename columns (since now its an np array)
df = pd.DataFrame(df)
df.columns = df_columns

df.head()
# visualise area-price relationship
sns.regplot(x="area", y="price", data=df, fit_reg=False)
# split into train and test
df_train, df_test = train_test_split(df,
                                     train_size = 0.7,
                                     test_size = 0.3,
                                     random_state = 10)
print(len(df_train))
print(len(df_test))
381
164
# split into X and y for both train and test sets
# reshaping is required since sklearn requires the data to be in shape
# (n, 1), not as a series of shape (n, )
X_train = df_train['area']
X_train = X_train.values.reshape(-1, 1)
y_train = df_train['price']

X_test = df_test['area']
X_test = X_test.values.reshape(-1, 1)
y_test = df_test['price']
```

OUTPUT:

RESULT

Thus the program for Linear Regression has been executed and verified successfully.

EX NO: 11 LOGISTIC REGRESSION

AIM :

To Perform Logistic Regression

ALGORITHM

Step1: Start

Step2: Import numpy,pandas,seaborn,matplotlib&sklearn

Step3: calculate logistic regression using the appropriate functions

Step4: display the result

Step 5: Stop

PROGRAM

```
portnumpy as np
import pandas as pd
importseaborn as sb
importmatplotlib.pyplot as plt
importsklearn
from pandas import Series, DataFrame
frompylab import rcParams
fromsklearn import preprocessing
fromsklearn.linear_model import LogisticRegression
fromsklearn.model_selection import train_test_split
fromsklearn import metrics
fromsklearn.metrics import classification_report
'exec(%matplotlib inline)'
rcParams['figure.figsize'] = 10, 8
sb.set_style('whitegrid')
url = 'https://raw.githubusercontent.com/BigDataGal/Python-for-Data-Science/master/titanic-
train.csv'
```

```
titanic = pd.read_csv(url)
titanic.columns =
['PassengerId','Survived','Pclass','Name','Sex','Age','SibSp','Parch','Ticket','Fare','Cabin','Embarke
d']
titanic.head()
sb.countplot(x='Survived',data=titanic, palette='hls')
titanic.isnull().sum()
titanic.info()
```

OUTPUT

RESULT

Thus the program for Logistics Regression has been executed and verified successfully.

EX NO: 12 TIME SERIES ANALYSIS

AIM :

To Perform Time series analysis

ALGORITHM

Step1: Start

Step2: Import numpy,pandas, matplotlib&seaborn

Step3: draw the plot

Step4: display the plot

Step 5: Stop

PROGRAM

```
fromdateutil.parser import parse

importmatplotlib as mpl

importmatplotlib.pyplot as plt

importseaborn as sns

importnumpy as np

import pandas as pd

plt.rcParams.update({'figure.figsize': (10, 7), 'figure.dpi': 120})

# Import as Dataframe

df = pd.read_csv('https://raw.githubusercontent.com/selva86/datasets/master/a10.csv',
parse_dates=['date'])

df.head()

# dataset source: https://github.com/rouseguy

df = pd.read_csv('https://raw.githubusercontent.com/selva86/datasets/master/MarketArrivals.csv')
```

```
df = df.loc[df.market=='MUMBAI', :]

df.head()

# Time series data source: fpppacakge in R.

importmatplotlib.pyplot as plt

df = pd.read_csv('https://raw.githubusercontent.com/selva86/datasets/master/a10.csv',
parse_dates=['date'], index_col='date')


# Draw Plot

defplot_df(df, x, y, title="", xlabel='Date', ylabel='Value', dpi=100):

    plt.figure(figsize=(16,5), dpi=dpi)

    plt.plot(x, y, color='tab:red')

    plt.gca().set(title=title, xlabel=xlabel, ylabel=ylabel)

    plt.show()

plot_df(df, x=df.index, y=df.value, title='Monthly anti-diabetic drug sales in Australia from 1992 to
2008.')
```

RESULT

Thus the program for Time series analysis has been executed and verified successfully.